

전자서명 (Digital Signatures) 발표 대본

Page 1. 표지

안녕하십니까. 이번 세미나 주제인 전자서명에 대해 발표하겠습니다.

Page 2. Plan

오늘 발표는 총 여섯 가지 주제로 구성되어 있습니다. 먼저 전자서명의 개념을 살펴보고, 보안 모델과 위조의 분류를 다룬 뒤, Plain RSA 서명과 그 취약점을 분석하겠습니다. 이어서 해시 기반 RSA 서명, DL 기반 서명을 설명하고, 마지막으로 무작위성과 해시의 중요성을 정리하겠습니다.

Page 3. 전자서명의 개념 (구분 슬라이드)

먼저 전자서명의 개념부터 설명드리겠습니다.

Page 4. 손글씨 서명의 한계

손으로 쓰는 물리적 서명은 문서에 대한 서명자의 동의를 증명하는 전통적인 수단입니다. 하지만 이를 단순히 디지털화할 경우 문제가 발생합니다. 서명을 스캔한 이미지는 단순히 복사하고 붙여넣는 것만으로도 쉽게 위조되거나 도용될 수 있기 때문입니다. 따라서 안전한 전자서명이 되기 위해서는 서명자가 누구인지뿐만 아니라, 서명 대상이 되는 메시지의 내용에도 반드시 의존해야 합니다.

Page 5. 손글씨 서명의 한계 (강조)

다시 한번 강조드리면, 동일한 서명자가 서명을 하더라도 대상 메시지가 달라지면 서명값 자체가 완전히 달라지도록 설계되어야 합니다. 이것이 전자서명이 갖추어야 할 핵심 조건입니다.

Page 6. 전자서명의 작동 원리

전자서명은 공개키와 비밀키로 이루어진 비대칭 키 쌍 구조를 기반으로 작동합니다. 서명을 생성할 때는 본인만 소유하고 있는 비밀키를 이용하여 해당 메시지에 고유한 서명을 부여합니다. 반대로 서명을 검증할 때는 공개된 검증키를 적용하여 해당 서명이 유효한지를 확인하게 됩니다.

Page 7. 전자서명의 작동 원리 (절차)

구체적인 절차는 다음과 같습니다. 첫째, 키 생성 알고리즘을 통해 검증키와 서명키 쌍을 생성하고 이를 분배합니다. 둘째, 서명할 메시지 M 을 준비합니다. 셋째, 비밀키를 이용하여 서명 시그마를 생성합니다. 넷째, 검증키를 이용하여 해당 메시지와 서명이 유효한지를 확인하고, 그 결과에 따라 수락 또는 거부를 결정합니다.

Page 8. 전자서명 vs MAC

전자서명을 메시지 인증 코드, 즉 MAC과 비교해보겠습니다. MAC은 대칭키 기반으로, 송신자와 수신자가 동일한 단일 키를 공유하는 방식입니다. 따라서 검증 서버가 해킹되어 키가 유출될 경우, 공격자가 완벽하게 서명을 위조할 수 있다는 약점이 있습니다. 반면 디지털 서명은 비대칭키 기반으로, 검증키와 서명키가 분리되어 있습니다. 그래서 서버가 해킹되어 검증키가 유출되더라도 서명 생성 자체는 불가능하기 때문에, 위조를 원천적으로 차단할 수 있습니다.

Page 9. 전자서명 vs MAC (표)

표로 정리하면, MAC은 대칭키 한 개를 사용하고 키 보유자만 검증할 수 있으며, 서버가 침해되면 위조가 가능합니다. 반면 전자서명은 키 쌍을 사용하고 누구나 검증이 가능하며, 서버가 침해되더라도 위조가 불가능합니다.

Page 10. Syntax

전자서명 스킴은 키 생성, 서명 생성, 서명 검증이라는 세 가지 다항시간 알고리즘으로 구성됩니다. 첫째, 키 생성 알고리즘은 검증키와 서명키 쌍을 생성합니다. 둘째, 서명 생성 알고리즘은 서명키와 메시지를 입력받아 서명을 출력합니다. 셋째, 서명 검증 알고리즘은 검증키, 메시지, 서명을 입력받아 유효하면 1을, 그렇지 않으면 0을 출력합니다.

Page 11. 정확성 요건

정확성 요건에 대해 설명드리겠습니다. 합법적인 사용자가 정상적인 비밀키로 생성한 서명은 검증 과정을 반드시 항상 통과해야 한다는 것이 기본 전제입니다. 키 생성이나 서명 과정에 무작위 요소가 섞여 매번 값이 달라지더라도, 최종 검증 결과는 항상 100퍼센트 수락되어야 합니다. 이러한 정확성 요건은 확률이 1이 되어야 한다는 수식으로 엄밀하게 정의됩니다.

Page 12. 보안 모델과 위조 분류 (구분 슬라이드)

다음으로 보안 모델과 위조의 분류에 대해 설명드리겠습니다.

Page 13. UF-CMA

전자서명의 대표적인 보안 모델인 UF-CMA를 소개하겠습니다. UF-CMA는 Chosen Message Attack, 즉 선택 메시지 공격 하에서의 Existential Unforgeability, 즉 존재적 위조 불가능성을 의미하는 모델입니다.

Page 14. UF-CMA (조건과 목표)

이 게임에서 공격자는 공개된 검증키를 보유하고 있으며, 원하는 임의의 메시지에 대해 마음껏 합법적인 서명을 요청하고 획득할 수 있습니다. 공격자가 승리하기 위해서는 서명을 요청한 적이 없는 새로운 메시지에 대해 유효한 서명을 제출해야 합니다. 즉, 공격자에게 가장 유리한 극한의 환경을 가정하더라도 새로운 서명 위조가 불가능함을 증명하는 것이 방어 목표입니다.

Page 15. 각 프로시저의 의미

이 게임은 서명의 안전성을 증명하기 위한 공격자와 방어자 간의 수학적 모델링입니다. Initialize 단계에서는 방어자 측이 키 쌍을 생성하고 공격자에게 공개키를 부여합니다. Sign 단계에서는 공격자가 원하는 메시지에 대해 오라클에 요청을 보내고, 이를 획득한 뒤 시스템이 요청 목록에 기록합니다. Finalize 단계에서는 공격자가 위조한 서명을 제출하면 시스템이 최종적으로 유효성을 판별하여 승패를 결정합니다.

Page 16. 승리 조건

공격자가 게임에서 승리하기 위해서는 두 가지 조건을 동시에 충족해야 합니다. 첫 번째는 Correct 조건으로, 제출한 서명이 검증 알고리즘을 정상적으로 통과해야 합니다. 두 번째는 New 조건으로, 제출한 메시지가 이전 서명 요청 목록에 없는 완전히 새로운 메시지여야 합니다.

Page 17. 승리 조건 (의미)

여기서 Sign 절차는 서명자를, Finalize 절차는 검증자를 표현합니다. 결국 이 게임은 공격자가 서명자가 이전에 서명하지 않은 메시지를 검증자가 수락하도록 만들 수 없음을 의미합니다.

Page 18. 서명 위조의 분류 체계

서명 위조는 공격이 달성하고자 하는 목표의 난이도에 따라 세 단계로 분류됩니다. Existential 위조는 공격자가 메시지와 서명 쌍을 자유롭게 선택할 수 있는 경우로, 가장 쉬운 난이도입니다. Selective 위조는 공격자가 오라클을 호출하기 전에 메시지를 미리 선택해야 하는 경우로, 중간 난이도입니다. Universal 위조는 공격자가 임의의 메시지에 서명할 수 있어야 하는 경우로, 가장 어려운 난이도입니다.

Page 19. 서명 위조의 분류 체계 (철학)

UF-CMA의 철학은 이 세 가지 위조 유형 중 가장 난이도가 낮은 Existential 위조조차 달성하지 못하도록 철저히 차단하는 것입니다.

Page 20. Plain RSA 서명과 취약점 (구분 슬라이드)

이어서 Plain RSA 서명과 그 취약점에 대해 살펴보겠습니다.

Page 21. Plain RSA 서명 스킴

Plain RSA 서명 스킴은 전통적인 RSA 암호 생성기를 아무런 변형 없이 서명 용도로 직접 활용하는 모델입니다. 서명을 만들 때는 서명자의 비밀키 지수를 메시지에 적용하여 모듈러 거듭제곱을 수행합니다. 이는 RSA의 역함수에 해당합니다. 서명을 검증할 때는 공개키 지수로 서명값을 거듭제곱하여 원래 메시지 값과 직접 비교하여 대조합니다.

Page 22. Plain RSA 서명 스킴 (알고리즘)

서명 알고리즘은 메시지 y 를 입력받아 비밀키 d 를 이용해 y 의 d 제곱을 N 으로 나눈 나머지를 서명 x 로 출력합니다. 검증 알고리즘은 메시지 y 와 서명 x 를 입력받아, x 의 e 제곱을 N 으로 나눈 나머지가 y 와 같은지를 확인합니다.

Page 23. 정확성 증명

정확성 증명은 오일러의 정리에 기반한 대수학적 특성을 활용합니다. 오일러 정리에 따르면 a 와 N 이 서로소일 때, a 의 $\phi(N)$ 제곱은 N 으로 나눈 나머지가 1이 됩니다. 그리고 e 와 d 사이에는 ed 가 1 더하기 k 곱하기 $\phi(N)$ 의 관계가 성립합니다. 이를 이용하면 x 의 e 제곱이 결국 y 와 같아짐을 증명할 수 있습니다.

Page 24. 예시1

간단한 예시를 보겠습니다. p 는 11, q 는 13으로 설정하면 N 은 143, $\phi(N)$ 은 120이 됩니다. e 를 7로 설정하면 120과 서로소이므로 유효합니다. 이때 d 는 7의 역원으로 103이 됩니다. 메시지 y 가 2일 때, 서명 x 는 2의 103제곱을 143으로 나눈 나머지인 6이 됩니다. 검증 단계에서 6의 7제곱을 143으로 나누면 다시 2가 나와, 검증이 성공함을 확인할 수 있습니다.

Page 25. 취약점1: Trivial Forgery

Plain RSA의 첫 번째 취약점은 Trivial Forgery, 즉 자명한 위조입니다. 공격자는 단순히 메시지 1과 서명값 1의 쌍을 검증자에게 제출하기만 하면 됩니다. 이는 1의 어떤 거듭제곱도 항상 1이라는 진리를 악용한 것입니다. 비밀키가 무엇이든 상관없이 1의 d 제곱은 항상 1이 되기 때문에 이 조건은 무조건 성립합니다. 그 결과 공격자는 서명 오라클을 단 한 번도 호출하지 않고도 완벽하게 위조 게임에서 승리할 수 있습니다.

Page 26. 취약점2: Multiplicative Homomorphism

두 번째 취약점은 곱셈 준동형성을 이용한 공격입니다. 공격자는 RSA의 대수적 특성인 곱셈 준동형성을 악용합니다. 절차는 다음과 같습니다. 임의의 두 메시지를 선택하여 각각에 대한 합법적인 서명값을 오라클로부터 획득합니다.

Page 27. 취약점2: Multiplicative Homomorphism (원리)

이렇게 획득한 두 서명값을 곱하기만 하면, 원본 메시지들의 곱에 대한 완벽한 합법 서명값으로 둔갑하게 됩니다. 이는 복잡해야 할 수학적 대수 구조가 시스템 겉면에 그대로 노출되어 있기 때문에 발생하는 취약점입니다.

Page 28. 취약점3: Inversion Attack

세 번째 취약점은 Inversion Attack, 즉 역산 공격입니다. 이 공격은 정상적인 메시지에서 서명을 생성하는 순서를 역전시킵니다. 절차는 다음과 같습니다. 먼저 무작위 숫자 x 를 선택하여 이를 서명값이라고 가정합니다. 그리고 공개키 e 를 이용해 역연산을 수행하여 결과값 y 를 강제로 생성합니다.

Page 29. 취약점3: Inversion Attack (원리)

이렇게 생성된 결과값 y 를 원본 메시지로, 처음의 무작위값 x 를 서명값으로 짜지어 제출합니다. 그 결과, 아무런 의미 없는 데이터라 하더라도 서명 오라클의 도움 없이 완벽하게 유효한 서명 쌍을 만들어낼 수 있습니다.

Page 30. 일방향성의 한계

이러한 취약점들이 발생하는 이유는 RSA의 일방향성만으로는 충분한 안전성을 보장할 수 없기 때문입니다. 일방향성 게임에서는 랜덤으로 주어지는 출력값에 대해 역추적이 어렵다는 것만을 보장할 뿐입니다.

Page 31. 일방향성의 한계 (UF-CMA와 비교)

하지만 UF-CMA 게임에서 서명 공격자는 무작위 값이 아니라, 스스로 조작하기 쉬운 특수한 형태의 메시지를 골라서 공격을 시도합니다.

Page 32. 일방향성의 한계 (결론1)

여기서 첫 번째 결론을 도출할 수 있습니다. 방어자가 제시하는 문제를 푸는 것, 즉 일방향성 게임과, 해커가 스스로 문제를 만들어내는 것, 즉 UF-CMA 게임은 그 차원이 완전히 다릅니다.

Page 33. 일방향성의 한계 (결론2)

두 번째 결론은 다음과 같습니다. 단순한 암호화 수준의 일방향성 전제만으로는 지능화된 서명 위조 공격을 방어할 수 없습니다.

Page 34. 해시 기반 RSA 서명 (구분 슬라이드)

이제 이러한 취약점을 보완한 해시 기반 RSA 서명에 대해 설명드리겠습니다.

Page 35. PKCS#1 서명 구조

Plain RSA의 문제를 완화하기 위해 PKCS#1 구조는 메시지를 특정 형태로 인코딩합니다. 거대 정수 공간의 원소가 아무 형태나 띠지 못하도록, 특정 비트 구조 패턴을 따르도록 강제하는 것입니다. 구체적으로는 메시지를 해시한 뒤, 고정된 패딩 바이트와 결합하여 최종적으로 RSA 연산을 적용합니다.

Page 36. PKCS#1 서명 구조 (수식)

N 이 1024비트, 즉 128바이트인 경우를 예로 들면, 패딩 함수는 고정 패턴과 해시값 20바이트를 결합하는 형태로 구성됩니다. 서명 생성식은 이렇게 패딩된 값에 비밀키 d 를 적용하여 N 으로 나눈 나머지를 구하는 방식입니다.

Page 37. PKCS#1의 한계

하지만 PKCS#1에는 구조적인 한계가 있습니다. 128바이트 중 무려 108바이트가 고정된 패턴이고, 오직 20바이트인 해시값만 변동됩니다. 이렇게 인위적으로 편향된 비트 구조 때문에 전체 모듈러 공간에서 고른 균등 분포를 달성하는 데 실패합니다. 그 결과, 균등 랜덤 입력을 전제로 하는 RSA 일방향성 모델로의 논리적 환원이 불가능하다는 증명상의 한계가 발생합니다.

Page 38. Full Domain Hash (FDH)

이러한 PKCS#1의 수학적 증명 불가 문제를 끊어낸 것이 1996년에 등장한 Full Domain Hash, 즉 FDH 서명 스킴입니다. 핵심 아이디어는 메시지의 특정 부분만 해싱하는 것이 아니라, 전체 정수 공간의 거대한 크기로 꽉 채워 확장하는 것입니다. 이렇게 공간을 가득 채운 거대한 해시값 위에 직접 RSA 역함수를 적용합니다. 그 결과 출력값이 전체 공간에 고르게 퍼지는 균등 분포를 달성하게 되어, 엄밀한 수학적 증명의 토대를 마련할 수 있습니다.

Page 39. Full Domain Hash (FDH) (알고리즘)

키 생성 알고리즘은 일반적인 RSA 키 생성과 동일합니다. 서명 알고리즘은 메시지를 해시하여 y 를 얻은 뒤, 비밀키로 거듭제곱하여 서명 x 를 생성합니다. 검증 알고리즘은 메시지를 다시 해시한 값과, 서명값을 공개키로 거듭제곱한 값이 일치하는지를 확인합니다.

Page 40. 충돌 저항성의 필요성

FDH가 안전하기 위해서는 기반이 되는 해시 함수에 충돌이 발생하지 않아야 합니다. 만약 해시 함수에 충돌이 발생하면 전체 방어 시스템이 즉시 붕괴할 수 있습니다. 공격 과정은 다음과 같습니다. 내용은 다르지만 해시값이 동일한 메시지 쌍을 탐색한 뒤, 한 메시지의 합법적인 서명을 다른 메시지의 서명으로 무단 복제하여 제출하는 방식입니다.

Page 41. 충돌 저항성의 필요성 (공격 코드)

구체적으로, 공격자는 해시 충돌 알고리즘을 이용해 메시지 쌍을 찾고, 첫 번째 메시지의 서명을 오라클로부터 획득한 뒤, 이를 두 번째 메시지의 위조 서명으로 제출합니다. 두 메시지의 해시값이 같기 때문에 서명값도 동일하게 성립합니다.

Page 42. 충돌 저항성의 필요성 (강조)

이를 통해 충돌 저항성이 서명의 안전성을 지키는 필수적인 전제 조건임을 알 수 있습니다.

Page 43. 충돌 저항성의 필요성 (한계)

다만 충돌 저항성은 필수조건이긴 하지만, 완전한 UF-CMA 방어를 위한 충분조건은 아니라는 점에 유의해야 합니다.

Page 44. Z_N 과 Z_N^* 의 차이

실제 구현에서는 SHAKE256과 같은 가변 출력 길이 함수를 이용하여 이론상 요구되는 긴 비트 수로 해시를 확장합니다. 이론상으로는 가역원 그룹의 출력이 필요하지만, 실제 구현은 전체 정수 범위로 출력을 도출합니다. 다만 출력값이 비가역원, 즉 p 나 q 의 배수로 맞아떨어질 확률은 무시할 수 있을 만큼 작습니다.

Page 45. Random Oracle 모델에서의 FDH 안전성

해시 함수 H 를 Random Oracle로 모델링할 경우, RSA FDH의 UF-CMA 안전성은 RSA의 일방향성으로 환원될 수 있습니다. 이때 공격자의 어드밴티지는 해시 쿼리 횟수와 RSA 일방향성 어드밴티지의 곱으로 제한됩니다. 문제는 이 쿼리 공격 횟수에 정비례하여 방어 시스템의 이론적 안전성이 깎여나가는 손실이 발생한다는 점입니다. 이러한 수학적 손실 문제는 이후 PSS라는 더 촘촘한 서명 스킴 연구의 기폭제가 됩니다.

Page 46. RSA PSS

RSA PSS는 FDH의 수학적 한계를 극복하고 등장한 규격으로, 현존 최고 수준의 확률론적 RSA 서명 스킴입니다. 가장 큰 차별점은 서명할 때마다 매번 다르게 생성되는 무작위 솔트 값을 투입한다는 것입니다.

Page 47. RSA PSS (구조)

RSA PSS는 세 단계의 해시 구역을 통해 원본 메시지와 솔트 값을 정교하게 엮고 마스킹하여 압축합니다. 이러한 방어 효과 덕분에 해커가 수집된 서명값을 역으로 분석하여 원본 메시지의 패턴이나 구조를 유추하려는 시도를 원천적으로 차단할 수 있습니다.

Page 48. 검증 알고리즘

RSA PSS의 검증 알고리즘은 다음과 같이 진행됩니다. 첫째, 서명 수신자는 검증키로 RSA 정방향 계산을 수행하여 겹겹이 쌓인 인코딩 데이터를 파싱하고 해체합니다. 둘째, 해시 함수의 자기 역원성, 즉 XOR 성질을 이용하여 마스킹된 겹질을 벗기고 순수한 솔트값을 색출합니다.

Page 49. 검증 알고리즘 (이어서)

셋째, 패리티 비트에 이상이 없는지 확인하여 최상위 비트가 0으로 세팅되어 있는지를 점검합니다. 넷째, 복원된 솔트와 제시된 메시지를 다시 결합하여 해싱한 값이 내부 w 구역과 100퍼센트 일치할 때만 서명을 최종적으로 수락합니다.

Page 50. PSS의 우수성

PSS는 증명 안전성을 깎아먹던 해시 쿼리 인수를 제거함으로써, 상대적으로 짧은 RSA 키로도 최상급의 방어력을 구현할 수 있습니다. 또한 동일한 원본 메시지가 하더라도 매번 전혀 다른 서명값을 생성하기 때문에 관찰 및 패턴 유추 공격을 무력화할 수 있습니다. 이러한 이론적 완벽함과 실용적 강점을 바탕으로, PSS는 현행 주요 국제 보안 프로토콜의 최고 규격으로 사용되고 있습니다.

Page 51. 표준화 목록

RSA PSS는 다양한 국제 표준에 채택되어 있습니다. RSA PKCS#1 v2.1, v2.2 및 RFC 8017, IEEE P1363a, ANSI X9.31, RFC 3447, ISO/IEC 9796-2 등이 그 대표적인 예입니다.

Page 52. DL 기반 서명 (구분 슬라이드)

이어서 이산로그 문제, 즉 DL 기반 서명에 대해 설명드리겠습니다.

Page 53. Schnorr 서명 스킴

Schnorr 서명 스킴은 무겁고 느린 소수 곱셈 기반의 RSA에서 탈피하여, 빠르고 가벼운 타원 곡선과 같은 순환 군 체계를 도입한 방식입니다. 공개키 X 가 g 의 x 제곱으로 주어질 때, 아무리 이를 들여다보아도 원래의 비밀키 스칼라 x 를 역추적하는 것이 불가능하다는 이산로그 문제, 즉 DLP를 이용합니다.

Page 54. Schnorr 서명 스킴 (절차)

Schnorr 서명의 절차는 다음과 같습니다. 첫째, 무작위 난수인 논스 r 를 생성하여 초기값을 발급하고, 이를 통해 공개 커밋먼트를 생성합니다. 둘째, 메시지와 커밋먼트를 섞어 해시 챌린지를 생성하고, 이를 이용하여 최종 서명 응답 s 를 신속하게 산출합니다.

Page 55. Schnorr 검증 알고리즘

Schnorr 검증 알고리즘은 다음과 같이 진행됩니다. 수신자는 건네받은 데이터인 R 과 메시지 M 을 기반으로 서명자가 생성했던 해시 챌린지 값 c 를 동일하게 재생산합니다. 그리고 서명자가 제출한 응답값을 이용해 g 의 s 제곱 결과를 좌변에 올리고, 우변에는 커밋먼트 R 과 챌린지를 섞은 공개키의 조합을 올려 양쪽이 정확히 일치하는지 대조합니다.

Page 56. Schnorr 검증 알고리즘 (정확성 증명)

정확성 증명 유도식을 살펴보면, g 의 s 제곱은 g 의 r 더하기 c 곱하기 x 제곱과 같고, 이는 다시 g 의 r 제곱과 g 의 x 제곱의 c 제곱의 곱, 즉 R 과 X 의 c 제곱의 곱으로 정리됩니다.

Page 57. Schnorr 검증 알고리즘 (의미)

이처럼 양변이 완벽히 일치하면, 진짜 비밀키 소유자만이 이 등식을 풀어 서명을 생성할 수 있었음이 입증됩니다.

Page 58. EdDSA

EdDSA는 이론적인 Schnorr 알고리즘을 실용성과 고속 처리 능력으로 끌어올린 타원 곡선 기반의 완성작입니다. 랜덤하게 발급된 키를 해시로 확장한 뒤, 구조적으로 가장 튼튼하고 안전한 위수를 가지도록 조각하는 작업을 수행합니다. 또한 외부 난수 생성기의 돌발적인 고장에 대비하여 무작위 논스를 사용하지 않습니다. 대신 본인만의 비밀키와 메시지 조각을 결합하여 흔들리지 않는 고유한 난수를 발생시키며, 이러한 특성 덕분에 최고 등급의 TLS 규격에 채택되었습니다.

Page 59. DSA / ECDSA

DSA는 1991년 NIST에 의해 제안되었으며, 기존의 El Gamal 기법을 고도화하여 정부 차원에서 빠르게 표준화한 방식입니다. 이후 무거운 유한체 기반의 DSA 코어를 덜어내고, 작고 빠른 타원 곡선 환경으로 포팅하여 이식한 것이 ECDSA입니다. ECDSA는 가벼운 용량과 빠른 처리 속도 덕분에 비트코인, 이더리움과 같은 블록체인 생태계의 지갑 및 자산 보호용 핵심 기술로 자리 잡았습니다.

Page 60. Schnorr vs ECDSA

Schnorr와 EdDSA를 ECDSA와 비교해보겠습니다. 서명 크기는 Ed25519 기준으로 두 알고리즘 모두 32바이트씩 두 개로 동일합니다. 다만 키 집약 가능 여부에서는 Schnorr 계열은 가능하지만 ECDSA는 불가능합니다. Tight security reduction, 즉 안전성 손실이 적은 증명 면에서도 Schnorr 계열은 존재하지만 ECDSA는 존재하지 않습니다. 결정론적 서명도 Schnorr 계열은 기본으로 제공하지만, ECDSA는 RFC 6979를 통해 별도로 구현해야 합니다. 특히 이슈는 Schnorr 계열에는 없으며, ECDSA는 현재는 만료된 상태입니다.

Page 61. 무작위성과 해싱 (구분 슬라이드)

마지막으로 무작위성과 해싱의 중요성에 대해 설명드리겠습니다.

Page 62. 서명에서의 무작위성

서명의 본질은 매번 형태를 감추어야 하는 암호화와는 다릅니다. 본인 인증을 목적으로 하는 서명은 굳이 무작위 패턴이 강제되지 않습니다. 다만 알고리즘의 태생상, DSA나 Schnorr 계열은 서명할 때마다 반드시 일회용 난수인 논스를 뽑아

야만 동작합니다.

Page 63. 예시2

구체적인 예시를 보겠습니다. 불량한 난수 생성기로 인해 서로 다른 두 문서에 동일한 논스가 재사용되는 순간, 간단한 뺄셈만으로 서명자의 비밀키가 1초 만에 유출될 수 있습니다. 논스 r 이 같을 경우, 두 서명식을 연립하면 다음과 같습니다. 첫 번째 서명식은 s_1 이 r 더하기 c_1 곱하기 x 이고, 두 번째 서명식은 s_2 가 r 더하기 c_2 곱하기 x 입니다. 두 식을 빼면 s_1 빼기 s_2 는 c_1 빼기 c_2 곱하기 x 가 되고, 이를 정리하면 비밀키 x 가 s_1 빼기 s_2 를 c_1 빼기 c_2 로 나눈 값으로 직접 노출됩니다.

Page 64. 해싱의 필수성1

해시 과정 없이 짧은 문자열 메시지 본문에 직접 거듭제곱하여 서명할 경우, 앞서 살펴본 Plain RSA와 동일한 공격이 그대로 작동합니다. 첫째, Trivial Forgery 공격으로, 입력값에 단순히 숫자 1만 입력해도 무조건 1로 출력되어 검증을 통과하게 됩니다. 둘째, Homomorphism 공격으로, 짧은 두 메시지의 서명값을 악의적으로 곱하더라도 모듈러 범위를 넘지 않기 때문에 곱셈 공격이 가능해집니다.

Page 65. 해싱의 필수성2

또 다른 예로, 강력한 암호학적 해시 대신 일반 통신망의 단순 오류 체크섬인 CRC16을 서명 패딩용으로 남용하는 경우를 살펴보겠습니다. CRC16은 16비트, 즉 65,536개라는 짧은 길이를 가지기 때문에 무차별 대입 공격을 시도하면 즉각적인 해시 충돌이 발생합니다. 또한 CRC는 단순한 선형 결합 구조이기 때문에, CRC 값이 0이 나오는 짝을 골라내어 곱셈으로 조작하면 방어벽이 쉽게 뚫리게 됩니다.

Page 66. 해싱의 필수성2 (결론)

이를 통해 다음과 같은 결론을 도출할 수 있습니다. 서명에서 검사 역할을 하는 함수는 반드시 암호학적 해시 함수로서의 성질을 갖추어야 합니다.

이상으로 전자서명에 대한 발표를 마치겠습니다. 감사합니다.